```python
#%%
#• FOR LOOP to parse all .log files in LOG_DIR and extract relevant data from each $DUTY line
# 'fname' is string variable (no readable data) in 'for' loop that stores filename of .log file
for fname in sorted(os.listdir(LOG_DIR)):
    # only process .log files
    if not fname.lower().endswith(".log"):
        continue
    # build full path to .log file
    log_path = os.path.join(LOG_DIR, fname)

    #• open .log file and read ("r") each line to extract relevant data from all $DUTY lines
    # 'f' is file object variable (readable data) in 'with' block of 'for' loop that represents .log file open/being read in current loop iteration
    with open(log_path, "r", encoding="utf-8", errors="replace") as f:
        # 'line' is string variable in inner 'for' loop (this 'for' loop is inside 'with' block that is inside main 'for' loop) that stores all text for an individual li
        for line in f:
            # remove invisible newline character ("\n") from end of each line so regex reads line as it actually is in .log file
            line = line.rstrip("\n")
            # use regex to extract string from lines that include "$DUTY"; if line fails regex, skip and move to next line
            match = re.search(r"\$DUTY,([^*]+)\*", line)
            if not match:
                continue
            # store capture group string extracted by regex in 'payload' variable
            payload = match.group(1)

            # Slice string stored in 'payload' variable into separate fields based on the position of the characters within the string
            start_sec_hex     = payload[0:8]
            start_milisec_hex = payload[8:12]
            end_sec_hex       = payload[12:20]
            end_milisec_hex   = payload[20:24]
            hydro_id          = payload[24:25]
            session_hex       = payload[25:33]
            samples_hex       = payload[33:41]

            # Convert values in hexadecimal format to UTC strings (for start/end date-time) or decimal integers (for sequential recording session number and count of sam
            start_utc = hex_seconds_to_utc(start_sec_hex)
            end_utc   = hex_seconds_to_utc(end_sec_hex)
            session_dec = int(session_hex, 16)
            samples_dec = int(samples_hex, 16)

            # Build 'log_rows' dictionary with column names as keys and data extracted from .log file $DUTY lines as values
            log_rows = {
                "LOG_FILE": fname,
                "RAW_LOG_LINE": line,
                "START_UTC": start_utc,
                "END_UTC": end_utc,
                "HYDROPHONE_ID": hydro_id,
                "RECORD_SESSION": session_dec,
                "N_SAMPLES_RECORDED": samples_dec
            }

            # if hydrophone ID in $DUTY line = "2" (low-frequency), append 'lf_log_rows' list with extracted data stored in 'log_rows' dictionary
            if hydro_id == "2":
                lf_log_rows.append(log_rows)
            # if hydrophone ID in $DUTY line = "3" (high-frequency), append 'hf_log_rows' list with extracted data stored in 'log_rows' dictionary
            elif hydro_id == "3":
                hf_log_rows.append(log_rows)
            # safety check: if hydrophone ID in $DUTY line does NOT equal 2 or 3, print warning message with relevent info
            else:
                print(
                    "[WARNING] Unexpected hydrophone ID found in $DUTY line\n"
                    f"  LOG FILE: {fname}\n"
                    f"  HYDROPHONE_ID: '{hydro_id}' (expected '2' or '3')\n"
                    f"  RAW LOG LINE: {line}\n"
                )
```

At the point in this FOR loop where you build the '**log_rows**' dictionary, the relevant section of the $DUTY line has been extracted, sliced into each individual variable (first 8 characters represent start date-time, etc.), and converted from hexadecimal format to UTC time or decimal number = **now we have the useable value for each variable:**

```
fname       = "UVI2_AroundSTX_20250306_001.log"
line        = "1741289626,692,$DUTY,...*74"
start_utc   = datetime(2025, 3, 6, 2, 1, 43, tzinfo=UTC)
end_utc     = datetime(2025, 3, 6, 3, 1, 43, tzinfo=UTC)
hydro_id    = "2"
session_dec = 12
samples_dec = 14400000
```

Dictionaries in python store things as:

```
{
  key: value,
  key: value,
  ...
}
```

Lists in python store things as:

```
[
  {row 1},
  {row 2},
  {row 3},
  ...
]
```

So when the code to build the '**log_rows**' dictionary is run…

```
log_rows = {
    "LOG_FILE": fname,
    "RAW_LOG_LINE": line,
    "START_UTC": start_utc,
    "END_UTC": end_utc,
    "HYDROPHONE_ID": hydro_id,
    "RECORD_SESSION": session_dec,
    "N_SAMPLES_RECORDED": samples_dec
}
```

… it stores the **variable names as keys** (in string format) and the **data for each variable as values** (in string, numeric integer, or datetime object format), creating:

```
log_rows = {
    "LOG_FILE": "UVI2_AroundSTX_20250306_001.log",
    "RAW_LOG_LINE": "1741289626,692,$DUTY,...*74",
    "START_UTC": datetime(2025, 3, 6, 2, 1, 43, tzinfo=UTC),
    "END_UTC": datetime(2025, 3, 6, 3, 1, 43, tzinfo=UTC),
    "HYDROPHONE_ID": "2",
    "RECORD_SESSION": 1,
    "N_SAMPLES_RECORDED": 14400000
}
```

↳ *which looks like this in table format:*

LOG_FILE	RAW_LOG_LINE	START_UTC	END_UTC	HYDROPHONE_ID	RECORD_SESSION	N_SAMPLES_RECORDED
UVI2_AroundSTX_20250306_001.log	1741289626,692,$DUTY,...*74	datetime(2025, 3, 6, 2, 1, 43, tzinfo=UTC)	datetime(2025, 3, 6, 3, 1, 43, tzinfo=UTC)	2	1	14400000

This **code is in a FOR loop**, so it creates and fills the dictionary for a **single individual $DUTY line** during each iteration of the loop.

In order to **permanently store the data for current $DUTY line after the next iteration of the FOR loop begins**, all of the **data stored in the '`log_rows`' dictionary for the current iteration is added to the '`lf_log_rows`' or '`hf_log_rows`' list** (based on the string value "2" or "3" of the "HYDROPHONE_ID" key) **as a new row** using **`append()`**:

```
if hydro_id == "2":
    lf_log_rows.append(log_rows)

if hydro_id == "3":
    hf_log_rows.append(log_rows)
```

The **data for each individual $DUTY line represent a single row in the list**, so after 2 iterations of the loop finish for $DUTY lines with low-frequency data, the '`hf_log_rows`' list will look like:

```
lf_log_rows = [
    {
        "LOG_FILE": "UVI2_AroundSTX_20250306_001.log",
        "RAW_LOG_LINE": "1741289626,692,$DUTY,...*74",
        "START_UTC": datetime(2025, 3, 6, 2, 1, 43, tzinfo=UTC),
        "END_UTC": datetime(2025, 3, 6, 3, 1, 43, tzinfo=UTC),
        "HYDROPHONE_ID": "2",
        "RECORD_SESSION": 1,
        "N_SAMPLES_RECORDED": 14400000
    },
    {
        "LOG_FILE": "UVI2_AroundSTX_20250306_002.log",
        "RAW_LOG_LINE": "1741318126,694,$DUTY,...*0A",
        "START_UTC": datetime(2025, 3, 6, 3, 1, 43, tzinfo=UTC),
        "END_UTC": datetime(2025, 3, 6, 4, 1, 43, tzinfo=UTC),
        "HYDROPHONE_ID": "2",
        "RECORD_SESSION": 2,
        "N_SAMPLES_RECORDED": 14400000
    },
]
```

↳ *which looks like this in table format:*

INDEX	LOG_FILE	RAW_LOG_LINE	START_UTC	END_UTC	HYDROPHONE_ID	RECORD_SESSION	N_SAMPLES_RECORDED
0	UVI2_AroundSTX_20250306_001.log	1741289626,692,$DUTY,...*74	datetime(2025, 3, 6, 2, 1, 43, tzinfo=UTC)	datetime(2025, 3, 6, 3, 1, 43, tzinfo=UTC)	2	1	14400000
1	UVI2_AroundSTX_20250306_002.log	1741318126,694,$DUTY,...*0A	datetime(2025, 3, 6, 3, 1, 43, tzinfo=UTC)	datetime(2025, 3, 6, 4, 1, 43, tzinfo=UTC)	2	2	14400000

Lists in python are zero-indexed, so the row containing data for the first $DUTY line appended to the list has index #0, and so on. *For example, the code `lf_log_rows[0]["LOG_FILE"]` would return `"UVI2_AroundSTX_20250307_001.log"`.*